Recipe Generation from Food Image

A PROJECT REPORT

Submitted by

NAME : A.PANDIYARAJ

REGNO : 0121127039

in partial fulfilment of the requirements

for the award of the degree

of

BACHELOR

OF

COMPUTER APPLICATIONS

Under the Guidance of

Mr. S.THIRUMARAN MCA., M.Phil., Ph.D.



DEPARTMENT OF COMPUTER APPLICATIONS

ALAGAPPA GOVERNMENT ARTS COLLEGE

KARAIKUDI-630 003.

(Grade- I college and Re-accredited with “B” Grade by NAAC)

(APRIL 2024)

ALAGAPPA GOVERNMENT ARTS COLLEGE, KARAIKUDI-630 003.

(Grade- I college and Re-accredited with "B" Grade by NAAC)

DEPARTMENT OF COMPUTER APPLICATIONS

BONAFIDE CERTIFICATE

This is to certify that the project entitled "**Recipe Generation from Food Image**" is the bonafide work done by **A.PANDIYARAJ , REG NO: 0121127039** who carried out the project work under my supervision and submitted during the academic year 2023-2024.

The Viva- Voce held on ……

INTERNAL EXAMINER **EXTERNAL EXAMINER**

HEAD OF THE DEPARTMENT

ACKNOWLEDGEMENT

I thank God, our almighty for his mercy to his towards me in concluding this project in a very successful way.

I wish to extend my thanks to our principal **Dr . A.PETHALAKSHMI M.Sc., M.Phil., Ph.D.,** Alagappa Government Arts College, Karaikudi who has provided all facilities are out the project successfully.

I wish to thank to **Dr . K.C. CHANDRASEKARAN M.Sc., M.Phil., PGDCA., Ph.D.,** our Head of the Department, Department of Computer Application, Alagappa Government Arts College, Karaikudi for approving and allowing us to carry out this project work.

I express my sincere thanks to my respected guide **Dr . S.THIRUMARAN MCA., M.Phil., Ph.D.,**Department of Computer Application, Alagappa Government Arts College, Karaikudi for her valuable suggestion and encouragement given throughout the project.

I would also like to thank all the staff members and Lab assistant of the Department of the Computer Application, who has helped us during the project.

I wish to express my heart full thanks to my parents who always encourage me in path of success.

Finally, I wish to thank our friends for their constant encouragement and valuable suggestions

TABLE OF CONTENTS

ACKNOWLEDGMENT

ABSTRACT PAGE

1. INTRODUCTION

1.1 Description 1
1.2 Existing system 2
1.3 Proposed system 3

2. SYSTEM DESIGN

2.1 Architectural design 5
2.2 Module description 15

3. SYSTEM IMPLEMENTATION 17

3.1 Software 18

4. **SYSTEM TESTING** 19

5. **FUTURE ENHANCEMENT** 21

6. **CONCLUSION** 22

REFRENCES 23

APPENDIX I

i) Hardware Requirements 24
ii) Software Requirements 24

APPENDIX II

iii) Sample screen shots 25
iv) Sample source coding 35

ABSTRACT

The "Recipe Generation from Food Image" project represents a groundbreaking fusion of artificial intelligence (AI) and culinary innovation, poised to redefine the way we approach cooking. This transformative system empowers users to effortlessly explore a world of culinary possibilities by simply uploading food images, initiating an automated recipe generation process.

At its core, the project consists of meticulously crafted frontend and backend components, seamlessly integrated to provide a user-friendly and efficient experience. The frontend interface enables intuitive image uploads, while the backend AI engine leverages advanced computer vision techniques, including cutting-edge web scraping technologies such as Google Lens and the ImgBB API, to extract intricate ingredient details from the uploaded images.

The resulting recipes are comprehensive and meticulously curated, complete with detailed ingredient lists, step-by-step cooking instructions, and essential nutritional information. Furthermore, the system goes beyond mere functionality by accommodating various dietary preferences, ensuring inclusivity and enhancing its overall user-centric design.

By pioneering the integration of AI and computer vision in recipe generation, this project embarks on a journey to inspire culinary creativity and exploration like never before. With its dynamic approach to personalized recipe discovery, it heralds a new era of culinary experiences, promising to spark a newfound passion for cooking among users worldwide.

1.INTRODUCTION

1.1 DESCRIPTION

In today's digital age, the convergence of artificial intelligence (AI) and culinary arts presents exciting opportunities for revolutionizing cooking experiences. Our project, titled "Recipe Generation from Food Image," aims to bridge the gap between visual recognition technology and culinary creativity by leveraging machine learning algorithms to automatically generate recipes from food images.

The project comprises two main components: a frontend user interface and a backend AI engine. The frontend interface allows users to upload images of dishes or ingredients, initiating the recipe generation process. Upon image submission, the backend AI engine employs advanced computer vision techniques to analyze and identify the contents of the image.

Once the image is recognized, the AI engine utilizes the ImgBB API for image uploading and processing. ImgBB provides a platform for efficient image hosting and allows seamless integration with our system for image recognition and analysis.

After analyzing the image, the AI engine retrieves relevant recipe data from online sources using web scraping techniques. Specifically, Google Lens web scraping is employed to extract detailed information about the dishes from the web.

The generated recipes are then presented to the user through the frontend interface, complete with detailed ingredient lists, cooking instructions, and nutritional information. Additionally, users have the option to explore alternative variations or similar recipes based on their preferences and dietary restrictions.

In conclusion, "Recipe Generation from Food Image" represents a pioneering initiative at the intersection of AI, web scraping, and gastronomy, offering a glimpse into the future of personalized and intelligent cooking experience

1.1 EXISTING SYSTEM

Existing System:

- **Manual Input and Textual Search:** Before the development of the "Recipe Generation from Food Image" project, traditional recipe discovery methods relied heavily on manual input and textual search queries. Users typically searched for recipes by entering keywords or browsing through extensive databases of recipes.
- **Lack of Visual Recognition:** Furthermore, existing recipe generation systems primarily focused on textual inputs, requiring users to manually input ingredients or select predefined options from drop-down menus. As a result, users were often required to manually transcribe or describe dishes, hindering the efficiency and convenience of the recipe discovery process.
- **Time-Consuming Process:** The manual nature of inputting textual data and navigating through extensive recipe databases made the recipe discovery process cumbersome and time-consuming for users.
- **Inability to Adapt and Evolve:** Without advanced AI capabilities, existing recipe generation systems lacked the ability to adapt and evolve based on user interactions and feedback. As a result, users were often presented with static and generic recipe recommendations, limiting the system's ability to cater to individual preferences and dietary requirements.
- **User Experience Limitations:** Overall, the existing system's reliance on manual input, lack of visual recognition capabilities, and limited personalization options resulted in suboptimal user experiences. These constraints underscored the need for a more advanced and user-centric solution capable of leveraging AI and computer vision technologies to enhance the recipe generation process and improve user satisfaction in the culinary domain.

DISADVANTAGES

- **Dependency on Image Quality:** The system's accuracy relies on clear and well-lit food images. Poor-quality images may lead to inaccuracies in ingredient recognition and recipe generation.
- **Limited Recipe Database:** Incomplete or outdated recipe repositories may restrict the variety of recipes generated, potentially resulting in repetitive or less relevant suggestions.
- **Inaccuracy in Ingredient Recognition:** Despite advancements, the system may struggle with accurately identifying ingredients from complex food images, leading to errors in generated recipes.
- **Difficulty in Handling Complex Dishes:** Generating recipes for intricate dishes with multiple components may pose challenges, impacting the system's ability to provide accurate cooking instructions.

1.2 PROPOSED SYSTEM

- **Image-Based Recipe Generation:** The proposed system introduces a novel approach to recipe discovery by leveraging advanced computer vision techniques to analyze food images. Users can upload images of dishes or ingredients, initiating the recipe generation process without the need for manual input.
- **Integration with Comprehensive Recipe Database:** To provide users with diverse and relevant recipe suggestions, the system integrates with a comprehensive recipe database sourced from reputable culinary sources and online repositories. This ensures access to a wide range of recipes tailored to various cuisines, dietary preferences, and cooking styles.
- **Personalization and Customization Options:** Recognizing the importance of catering to individual preferences and dietary restrictions, the system offers personalized recipe recommendations based on user preferences. Users can customize their recipe preferences, including dietary restrictions, preferred cuisines, and ingredient preferences, to receive tailored recipe suggestions aligned with their culinary preferences.
- **Dynamic User Interface:** The system features a user-friendly interface designed to streamline the recipe discovery process and enhance user engagement. Intuitive navigation and interactive features facilitate seamless interaction with the system, empowering users to explore and discover new culinary inspirations effortlessly.
- **Continuous Improvement through User Feedback:** To ensure ongoing refinement and optimization, the system incorporates mechanisms for collecting user feedback and preferences. By analyzing user interactions and feedback, the system can adapt and improve its recipe recommendations over time, enhancing user satisfaction and overall system performance.
- **Efficient Integration of External APIs:** The proposed system efficiently integrates with external APIs, including image recognition services and recipe databases, to enhance functionality and performance.
- **Advanced Image Analysis Algorithms:** A primary focus of the proposed system involves the integration of cutting-edge image processing algorithms, leveraging sophisticated computer vision techniques and machine learning models. These

advancements will significantly enhance the accuracy and efficiency of ingredient extraction from food images, resulting in more precise recipe generation.

- **Revamped User Interface and Experience:** A comprehensive overhaul of the user interface (UI) is planned to provide users with a visually appealing and intuitive platform. The revamped UI will feature streamlined navigation, interactive elements, and responsive design to ensure seamless interaction across devices and screen sizes.
- **Expanded Recipe Database and Diversity:** The proposed system will integrate additional recipe databases and APIs to augment the variety and diversity of recipes available to users. By curating a vast collection of recipes spanning various cuisines, dietary preferences, and cooking styles, the platform will cater to a broader audience and offer users a richer culinary experience.
- **Mobile Optimization and Native Applications:** The proposed system will include the development of native mobile applications for iOS and Android platforms, offering users a seamless and optimized mobile experience. These applications will leverage platform-specific capabilities and provide offline access to recipes, enabling users to access their favorite recipes anytime, anywhere.
- **Comprehensive Nutritional Analysis:** The nutritional analysis capabilities of the platform will be expanded to provide users with detailed insights into the nutritional content of their meals. By incorporating comprehensive nutritional data sources and adhering to FDA guidelines, the platform will offer users accurate and reliable nutritional information for informed meal planning.

ADVANTAGES

- **Effortless Recipe Discovery:** The project offers users a seamless and intuitive way to discover new recipes by simply uploading images of dishes or ingredients, eliminating the need for manual input or extensive recipe searches.
- **Enhanced User Engagement:** By leveraging visual recognition technology, the project enhances user engagement by providing a more interactive and visually appealing experience compared to traditional text-based recipe platforms.
- **Personalized Recommendations:** The system's ability to analyze user preferences and dietary restrictions enables personalized recipe recommendations, catering to individual tastes and dietary needs.
- **Diverse Culinary Exploration:** Integration with a comprehensive recipe database ensures access to a wide range of culinary options from various cuisines and cooking styles, encouraging users to explore and experiment with diverse dishes.
- **Time-Saving Solution:** By automating the recipe generation process, the project saves users time and effort, allowing them to quickly discover new recipes and culinary inspirations without extensive manual input.
- **Continuous Improvement and Adaptation:** The system's ability to analyze user feedback and adapt recipe recommendations over time ensures continuous improvement and optimization, enhancing user satisfaction and overall system performance.
- **Convenient and User-Friendly Interface:** The project features a user-friendly interface designed to streamline the recipe discovery process, making it accessible to users of all skill levels and culinary backgrounds.
- **Empowerment of Culinary Creativity:** By providing users with tailored recipe recommendations and encouraging exploration, the project empowers culinary enthusiasts to unleash their creativity and experiment with new ingredients and cooking techniques.

2. SYSTEM DESIGN

2.1 ARCHITECTURAL DESIGN:

The architectural design of the "Recipe Generation from Food Image" project encompasses the following components and their interactions:

1. **Frontend Interface:** The frontend interface serves as the user-facing component of the system, allowing users to interact with the application. It includes features such as image upload functionality, user authentication, and recipe display. Built using HTML, CSS, and JavaScript frameworks , the frontend interface provides an intuitive and responsive user experience.
2. **Backend Server:** The backend server acts as the central processing unit of the system, handling user requests, image processing, and recipe generation. It is implemented using server-side technologies such as Node.js , providing a robust and scalable infrastructure for handling concurrent user interactions.
3. **Image Processing Module:** The image processing module is responsible for analyzing uploaded food images and extracting relevant information, such as ingredient identification and dish recognition.
4. **Recipe Generation Engine:** The recipe generation engine generates recipes based on the information extracted from the uploaded images. It accesses a comprehensive recipe database to retrieve relevant recipe data and formats it into user-readable recipes.
5. **External API Integration:** The system integrates with external APIs, such as image recognition services and recipe databases, to enhance functionality and access additional resources. APIs like ImgBB API and Google Lens web scraping method for image recognition and Edamam API for recipe data retrieval provide essential functionalities for image processing and recipe generation.
6. **Database Management System (DBMS):** The DBMS stores user data, and system configurations. It ensures data integrity, security, and efficient data retrieval for various system functionalities. Commonly used databases like XEMPP ,SQL are employed to manage structured and unstructured data efficiently.

7. **Authentication and Authorization Module:** The authentication and authorization module handles user authentication, access control, and session management. It ensures that only authorized users can access specific functionalities of the system, enhancing security and protecting user data.
8. **API Gateway Integration:** The system includes an API gateway to streamline communication between microservices, acting as a centralized entry point for requests. This promotes modularity and simplifies management, ensuring efficient interactions within the system.
9. **User Feedback and Analytics:** The system includes modules for collecting user feedback and analytics data to continuously improve recipe recommendations and system performance. User interactions, preferences, and feedback are analyzed to refine recipe generation algorithms and enhance user satisfaction.

Overall, the architectural design of the system prioritizes modularity, scalability, and extensibility, allowing for easy integration of new features and technologies while ensuring optimal performance and user experience.

MODULE DESCRIPTION

- **User Interface Module:** This module provides the user-facing interface for interacting with the system. It includes features for uploading food images, viewing generated recipes, and customizing user preferences.
- **Image Processing Module:** Responsible for processing uploaded food images, this module utilizes computer vision algorithms to identify ingredients and extract relevant information necessary for recipe generation.
- **Recipe Generation Module:** This module generates recipes based on the information extracted from the uploaded images. It accesses a comprehensive recipe database to retrieve relevant recipe data and formats it into user-readable recipes.
- **External API Integration Module:** Integrates with external APIs, such as image recognition services and recipe databases, to enhance functionality and access additional resources required for image processing and recipe generation.
- **Authentication and Authorization Module:** Handles user authentication, access control, and session management to ensure secure access to system functionalities and protect user data.
- **Feedback and Analytics Module:** Collects user feedback and analytics data to refine recipe recommendations and improve system performance over time. It analyzes user interactions, preferences, and feedback to enhance the relevance and accuracy of generated recipes.
- **Database Management Module:** Manages the storage and retrieval of user data, recipe information, and system configurations. It ensures data integrity, security, and efficient data retrieval for various system functionalities.
- **Logging and Monitoring Module:** Monitors system performance, tracks user interactions, and logs system events and errors for troubleshooting and performance optimization purposes. It provides insights into system health and user behavior to ensure optimal system operation.
- **Web Scraping :** This module retrieves recipes from the web based on the identified food items from Google Lens. It likely involves parsing HTML content from recipe websites to extract relevant information.

- **User Profile Management Module:** This module allows users to create and manage their profiles within the system. It enables users to update personal information, manage dietary preferences, and track their recipe history.
- **Google Lens Integration:** This module interacts with Google Lens API. It sends the preprocessed image data to Google Lens and retrieves the identified food items within the image.
- **Error Handling and Exception Management Module:** Responsible for handling errors and exceptions that may occur during system operation. This module ensures graceful degradation and recovery from errors, maintaining system stability and reliability. It logs error details and provides appropriate error messages to users for troubleshooting purposes.

3.SYSTEM IMPLEMENTATION

- **Frontend Development:** The frontend of the project is developed using HTML, CSS, and JavaScript. It includes components for user interaction, such as uploading food images and displaying recipe results. The frontend design is intuitive and user-friendly, providing a seamless experience for users.
- **Backend Development with Node.js:** The backend of the project is implemented using Node.js, a runtime environment for executing JavaScript code server-side. Node.js facilitates asynchronous event-driven programming, making it well-suited for handling concurrent requests in a non-blocking manner. It includes modules for image processing, API integration, and database management.
- **Image Processing with Google Lens:** Image processing functionality is integrated using Google Lens, a powerful image recognition tool. Google Lens analyzes uploaded food images to identify ingredients and extract relevant information necessary for recipe generation. This integration enhances the accuracy and efficiency of the system's image processing capabilities.
- **API Integration:** The project integrates with external APIs, including the Edamam API for recipe data retrieval and the imgbb API for image uploading and hosting. These APIs provide essential functionalities for accessing recipe data, processing images, and enhancing system functionality.
- **Database Management with XAMPP:** XAMPP is utilized for database management, providing an easy-to-use solution for running a MySQL database server locally. The database stores user data, recipe information, and system configurations, ensuring data integrity and efficient data retrieval for various system functionalities.
- **Authentication and Authorization:** User authentication and authorization functionalities are implemented to secure access to system features and protect user data. This includes mechanisms for user registration, login authentication, and access control.
- **Testing and Quality Assurance:** Rigorous testing processes, including unit testing and integration testing, are conducted to ensure the reliability and stability of the system. Quality assurance measures are implemented to identify and address any issues or bugs during development and deployment.

3.1 SOFTWARE

- **HTML, CSS, and JavaScript:**
    1. HTML (Hypertext Markup Language) provides the structure and content of web pages.
    2. CSS (Cascading Style Sheets) is used for styling and layout, controlling the presentation and appearance of web pages.
    3. JavaScript is a programming language that adds interactivity and dynamic behavior to web pages, enabling features such as form validation, user input handling, and DOM manipulation.
- **Node.js:**
    1. Node.js is a runtime environment that allows you to run JavaScript code on the server side. It is built on the V8 JavaScript engine and provides a non-blocking, event-driven architecture, making it efficient for handling concurrent requests.
    2. Node.js is commonly used for backend development, handling server-side logic, database operations, and API integrations.
- **XAMPP:**
    1. XAMPP is a free and open-source cross-platform web server solution package.
    2. It includes Apache, MySQL, PHP, and Perl.
- **Google Lens:**
    1. Google Lens is an image recognition tool developed by Google.
    2. In this project, Google Lens is utilized for image recognition, specifically for analyzing uploaded food images to identify food name
- **Edamam API:**
    1. The Edamam API is a food and recipe data API that provides access to a vast database of recipes, nutrition information, and ingredient data.
    2. It offers endpoints for searching and retrieving recipe data based on various parameters such as ingredients, dietary preferences, and cuisines. In this project, the Edamam API is integrated to fetch recipe information based on user queries, enhancing the system's ability to generate recipes from food images.
- **imgbb API:**
    1. The imgbb API is a service for image uploading and hosting.It allows users to upload images to the imgbb platform and provides URLs for accessing the uploaded images.

4. SYSTEM TESTING

- **Unit Testing:**
    1. **Frontend Unit Testing:** Conduct comprehensive tests on individual frontend components to ensure responsiveness, usability, and compatibility across various devices and screen sizes. Test individual components of the frontend interface to ensure they function as expected.
    2. **Backend Unit Testing:** Perform rigorous testing on backend modules to ensure efficiency, reliability, and robustness in handling data processing, API interactions, and server-side logic. Test individual modules of the backend server, including image processing, API integration, and database management, to verify their functionality and behaviour .
- **Integration Testing:**
    1. **Frontend-Backend Integration Testing:** Validate seamless integration between frontend and backend components, ensuring efficient communication, data exchange, and synchronization for a smooth user experience. Test the integration between the frontend and backend components to ensure seamless communication and data exchange. This includes testing API endpoints, data retrieval, and response handling.
    2. **External API Integration Testing:** Verify the reliability, accuracy, and responsiveness of external API integrations such as the Edamam API and imgbb API to enhance system functionality and performance. Test the integration with external APIs such as the Edamam API and imgbb API to verify proper data exchange and functionality.
- **User Acceptance Testing (UAT):**
    1. **Functional Testing:** Conduct functional testing to verify that the system meets user requirements and performs as expected. This includes testing features such as image uploading, recipe generation, and user authentication.
    2. **Usability Testing:** Evaluate the user interface for ease of use, intuitiveness, and accessibility. Gather feedback from users to identify areas for improvement and enhance the overall user experience.

- **Performance Testing:**
    1. **Load Testing:** Simulate a high volume of user traffic to assess system performance under load. Measure response times, throughput, and resource utilization to ensure the system can handle peak loads without degradation in performance.
    2. **Stress Testing:** Apply stress to the system beyond its normal operating conditions to identify its breaking point and determine how it behaves under extreme stress.
- **Security Testing:**
    1. **Vulnerability Assessment:** Identify and address potential security vulnerabilities such as SQL injection, cross-site scripting (XSS), and authentication bypass vulnerabilities. Implement security measures such as input validation, parameterized queries, and encryption to protect against security threats.
    2. **Authentication and Authorization Testing:** Test user authentication and authorization mechanisms to ensure only authorized users have access to system features and data.
- **Compatibility Testing:**
    1. **Browser Compatibility Testing:** Test the application across different web browsers (e.g., Chrome, Firefox, Safari) to ensure compatibility and consistent behavior.
    2. **Device Compatibility Testing:** Test the application on different devices (e.g., desktops, tablets, smartphones) to ensure responsiveness and compatibility across various screen sizes and resolutions.
- **Regression Testing:**
    1. **Regression Test Suites:** Execute regression test suites to verify that recent code changes have not introduced new defects or caused regression in existing functionality. This ensures that the system remains stable and reliable after updates or modifications.
- By conducting thorough system testing across these various dimensions, the "Recipe Generation from Food Image" project can ensure the reliability, performance, security, and usability of the system, ultimately delivering a high-quality user experience.

5. FUTURE ENHANCEMENT

- **Enhanced Image Recognition Models:**
    1. Incorporate state-of-the-art deep learning models for image recognition to improve accuracy and expand the range of recognized ingredients and dishes.
    2. Explore the use of transfer learning and fine-tuning techniques to adapt pre-trained models to specific food recognition tasks.
- **Augmented Reality Integration:**
    1. Implement augmented reality (AR) features that allow users to overlay recipe instructions and cooking tips directly onto their surroundings using their device's camera.
    2. Enable users to visualize cooking steps in real-time, enhancing the cooking experience and providing interactive guidance.
- **Voice-Activated Assistance:**
    1. Integrate voice-activated commands and virtual assistants to provide hands-free interaction with the system.
    2. Allow users to verbally request recipe suggestions, receive cooking instructions, and navigate through the application using natural language commands.
- **Smart Ingredient Substitution:**
    1. Implement intelligent ingredient substitution algorithms that suggest alternative ingredients based on user preferences, dietary restrictions, and ingredient availability.
    2. Utilize machine learning algorithms to analyze ingredient properties and recommend suitable substitutions while preserving the flavor and nutritional balance of recipes.
- **Personalized Nutrition Tracking:**
    1. Enhance the system with personalized nutrition tracking features that allow users to track their daily intake of nutrients, calories, and macronutrients.
    2. Provide insights and recommendations for achieving nutritional goals and maintaining a balanced diet based on individual preferences and dietary requirements.

- **Community-Based Recipe Sharing:**
    1. Create a community platform where users can share their own recipes, cooking tips, and culinary experiences with others.
    2. Implement social networking features such as user profiles, recipe reviews, and recipe sharing to foster a vibrant community of food enthusiasts.
- **Machine Learning-Powered Recipe Recommendations:**
    1. Develop advanced recommendation algorithms that leverage machine learning techniques to analyze user preferences, cooking habits, and feedback.
    2. Provide personalized recipe recommendations tailored to each user's taste preferences, cooking skill level, and dietary preferences.
- **Cross-Platform Compatibility:**
    1. Extend the application's compatibility to multiple platforms, including mobile devices, tablets, smart TVs, and smart kitchen appliances.
    2. Ensure seamless synchronization of user data and preferences across different devices to provide a consistent and cohesive user experience.
- By implementing these future enhancements, the "Recipe Generation from Food Image" project can stay at the forefront of innovation, provide valuable features for users, and continue to evolve and adapt to changing user needs and technological advancements in the field of culinary technolog

6.CONCLUSION

The "Recipe Generation from Food Image" project embodies a paradigm shift in culinary technology, offering users a seamless and efficient way to explore, discover, and create delicious recipes from food images. Throughout the development journey, our focus has been on delivering a highly responsive and efficient system that caters to the diverse needs and preferences of our users.

By harnessing the power of image recognition technology and integrating with external APIs such as the Edamam API and imgbb API, we have created a dynamic platform that empowers users to effortlessly convert food images into inspiring recipes. Our commitment to responsiveness is evident in the intuitive user interface, which adapts seamlessly to various devices and screen sizes, ensuring an optimal user experience regardless of the platform used.

"Recipe Generation from Food Image" project is more than just a technological marvel it is a testament to our commitment to excellence and our passion for transforming the way we interact with food. With every recipe generated and every culinary adventure embarked upon, we reaffirm our belief that the intersection of technology and gastronomy holds endless possibilities for creativity, discovery, and delight.

In conclusion, the "Recipe Generation from Food Image" project stands as a testament to our unwavering commitment to responsiveness, efficiency, and user satisfaction. As we continue to innovate and evolve, we remain dedicated to providing our users with a truly transformative culinary experience that inspires creativity, simplifies meal planning, and fosters a deeper connection to the world of food

REFERENCES

[1]. Huang, T. S., & Faerber, G. (2019). Visual Content Indexing and Retrieval with Psycho-Visual Models (2nd ed.). Springer.

[2]. Li, H., & Jain, A. K. (2011). Handbook of Face Recognition. Springer.

[3]. Gonzalez, R. C., Woods, R. E., & Eddins, S. L. (2018). Digital Image Processing Using MATLAB (4th ed.). Gatesmark Publishing.

[4]. Ong, K. E., & White, J. (2019). Full-Stack Vue.js 2 and Node.js: Build scalable and powerful web apps with modern web stack: Vue.js 2, Node.js, MongoDB, and Express.js. Packt Publishing.

[5]. Brown, C. (2018). Web Development with MongoDB and Node.js (3rd ed.). Packt Publishing.

[6]. Mehta, N. (2018). Learning Node.js Development: Learn the fundamentals of Node.js, and deploy and test Node.js applications on the web. Packt Publishing.

[7]. XAMPP. "XAMPP." Accessed from https://www.apachefriends.org/index.html.

[8]. Edamam. "Edamam API." Accessed from https://developer.edamam.com/edamam recipe-api.

[9]. imgbb. "imgbb API." Accessed from https://api.imgbb.com/.

[10]. Node.js. "Node.js." Accessed from https://nodejs.org/.

[11]. Reference “youtube link” Accessed from https://www.youtube.com/watch?v=HjakA1gFfh8&t=1129s].

APPENDIX I

The following will be needed in order to complete this project

i) HARDWARE REQUIREMENTS

- Processor : Intel Core i3 or Above AMD Ryzen series or Above
- RAM : 4GB DDR4 or Higher
- Hard Disk : 256 GB solid-state drive (SSD) or Higher
- Network Interface : Gigabit Ethernet or 802.11ac Wi-Fi
- Monitor : 1920x1080 display resolution or Higher

ii) SOFTWARE REQUIREMENTS

- Front End : HTML, CSS, Java Script
- Back End : Node JS
  Puppeteer,
  Express.js
  XAMPP,
  MYSQL
- API Integration : Edamam API ,
  ImgBB API ,
  Google Lens

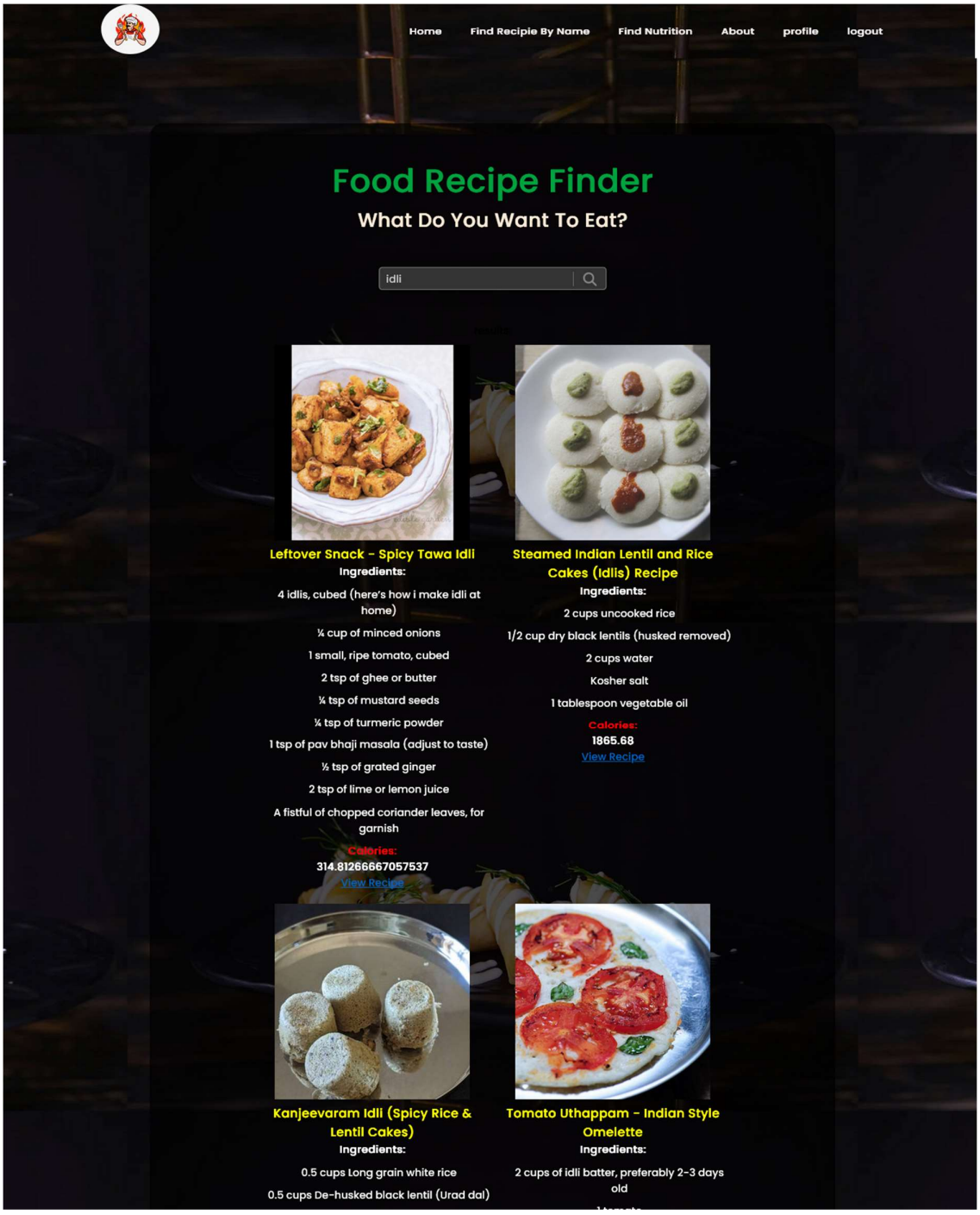
APPENDIX II

iii) SAMPLE SCREENSHOT

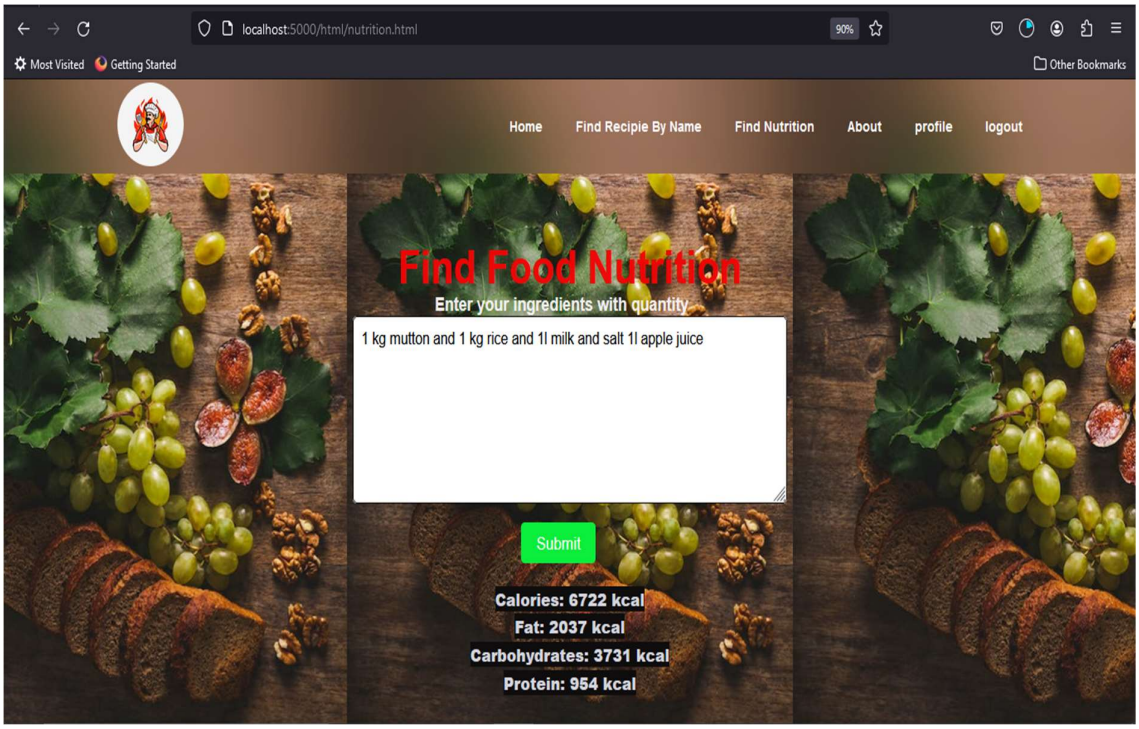
HOME PAGE:



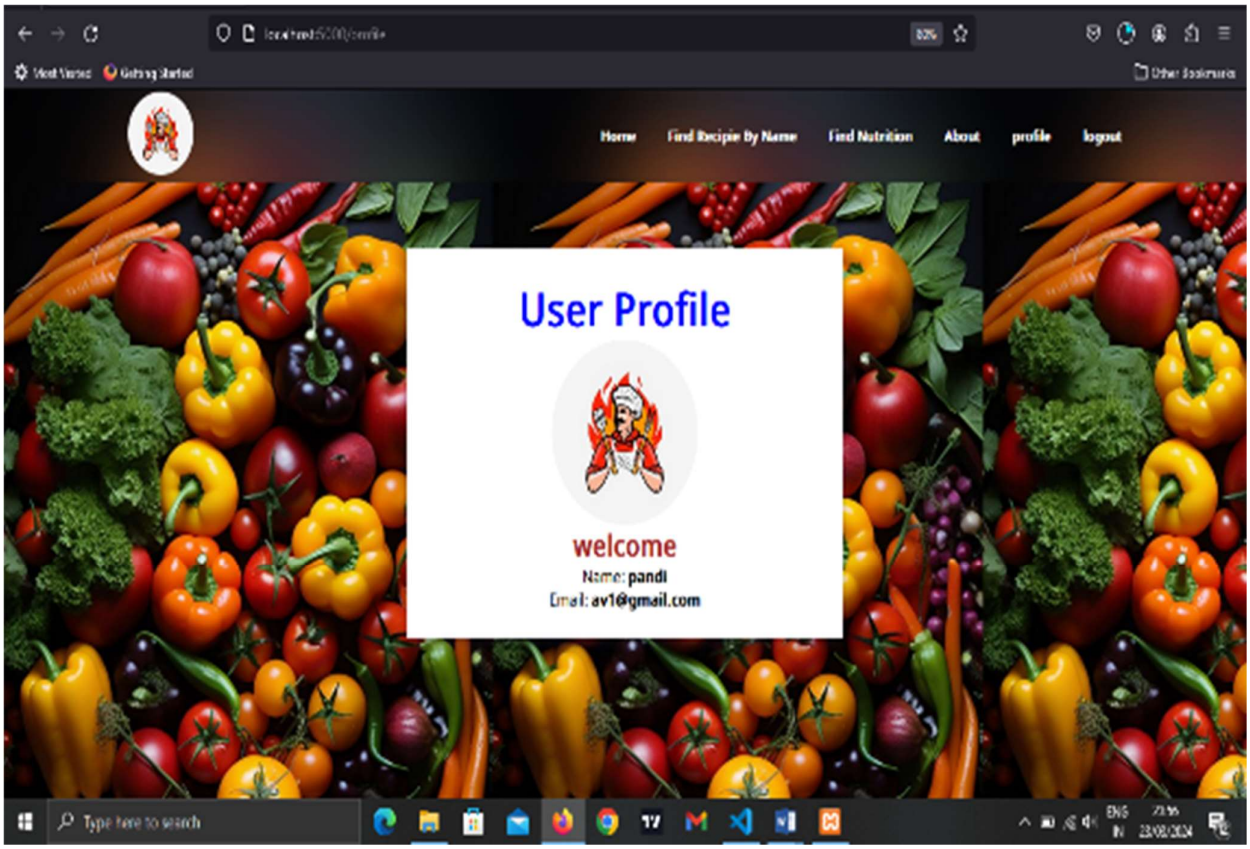
FIND RECIPE BY NAME PAGE:



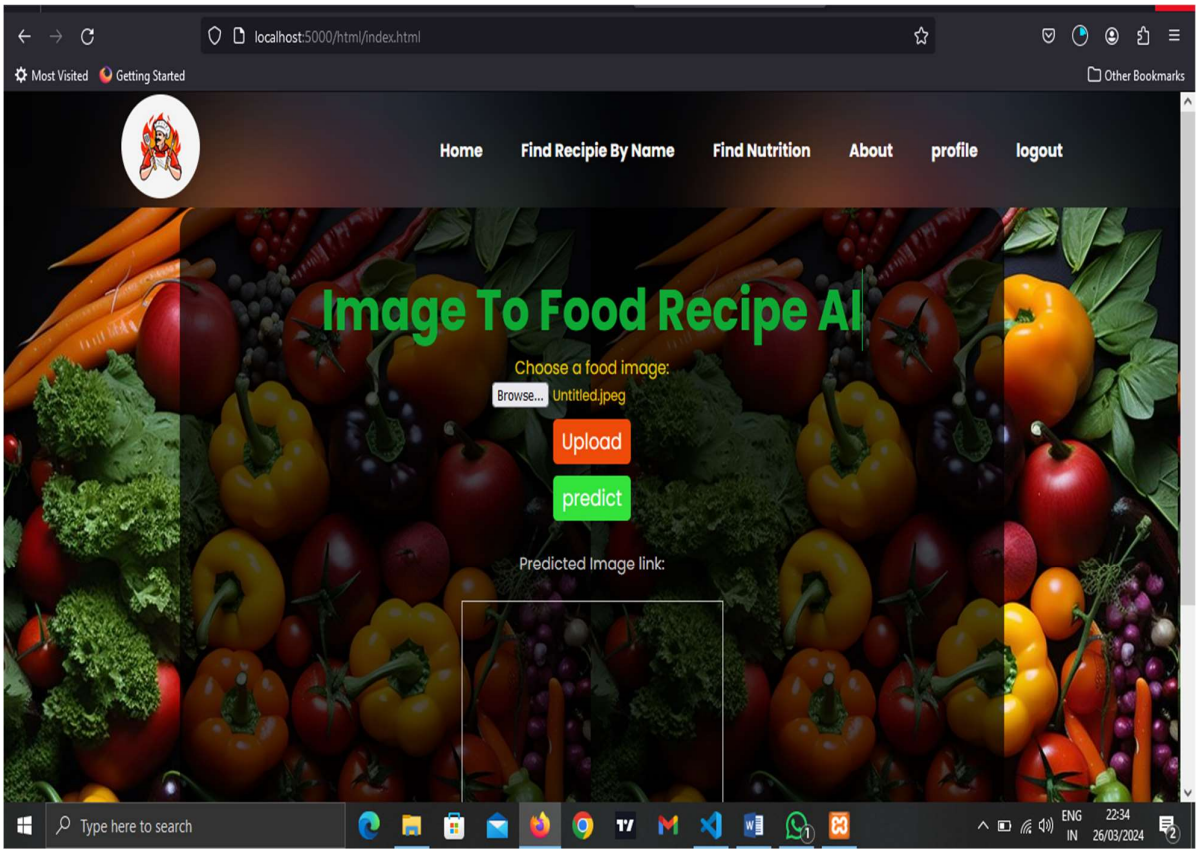
FIND NUTRITION PAGE :



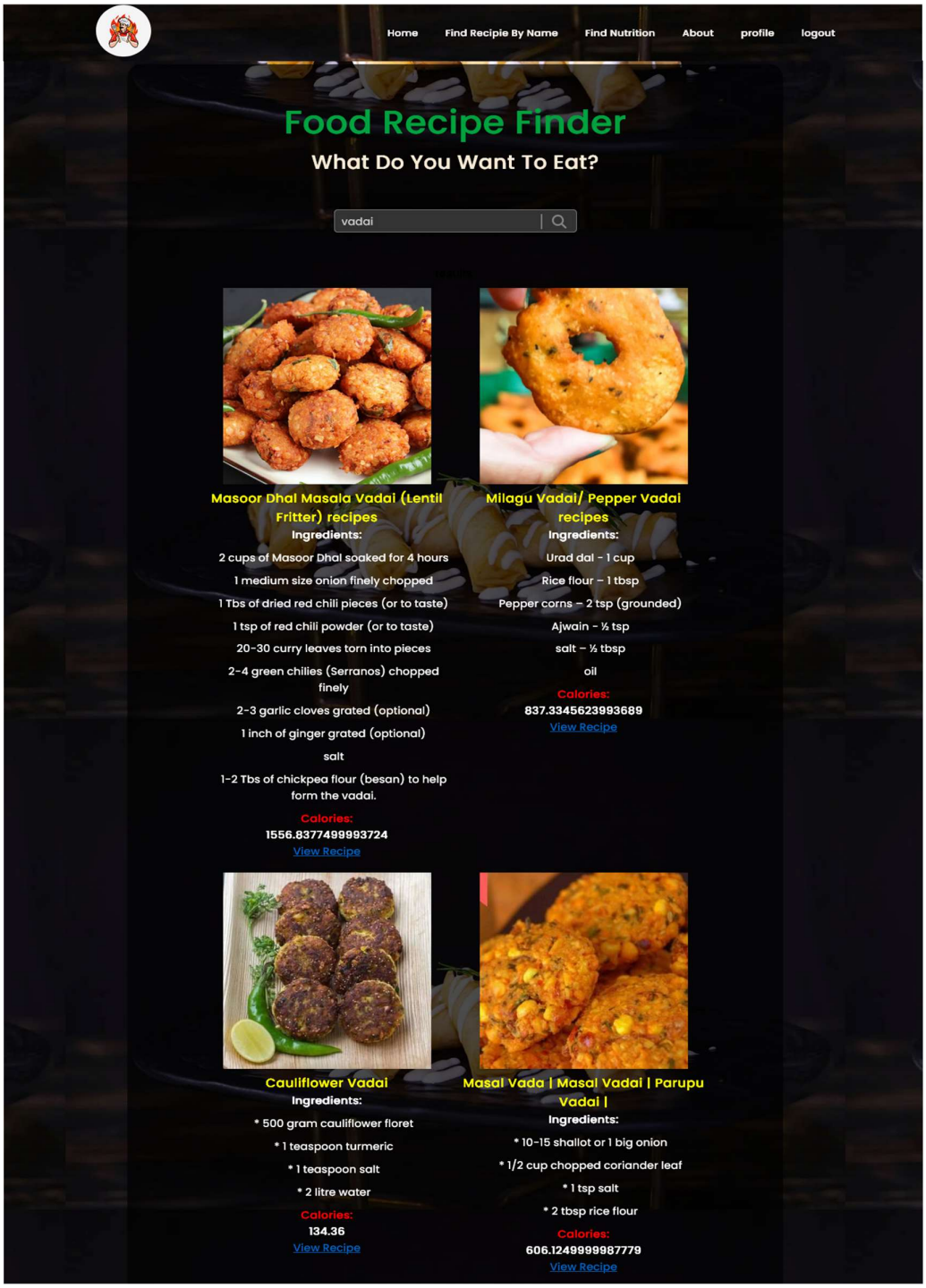
USER PROFILE :



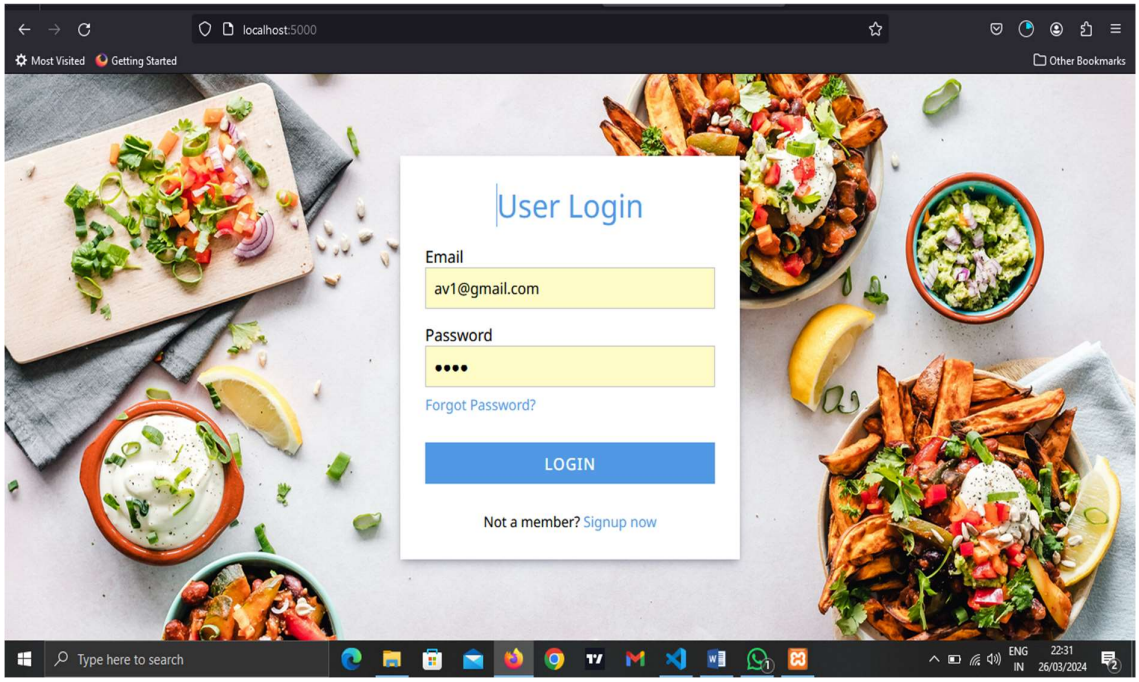
HOME PAGE:


localhost:5000/html/index.html
Most Visited
Getting Started
Other Bookmarks
Home
Find Recipie By Name
Find Nutrition
About
profile
logout
Image To Food Recipe AI
Choose a food image:
Browse...
Untitled.jpeg
Upload
predict
Predicted Image link:
Type here to search
ENG
IN
22:34
26/03/2024

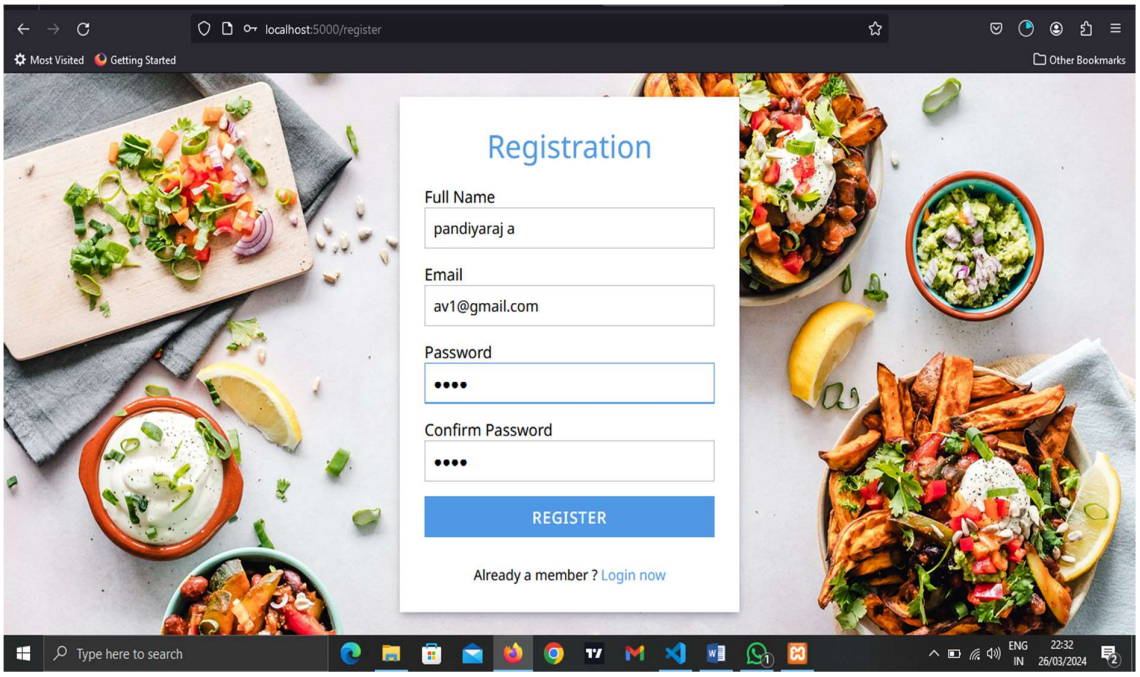
FIND RECIPE BY NAME PAGE:



LOGIN PAGE:


localhost:5000
Most Visited
Getting Started
Other Bookmarks
User Login
Email
av1@gmail.com
Password
••••
Forgot Password?
LOGIN
Not a member? Signup now
Type here to search
ENG
IN
22:31
26/03/2024

REGISTRATION PAGE:



iv)SAMPLE SOURCE CODING

Under Project Folder Create a Node js file named app.js

```
const express = require("express");
const mysql = require("mysql");
const dotenv = require("dotenv");
const path = require("path");
const hbs = require("hbs");
const cookieParser = require("cookie-parser");
const puppeteer = require('puppeteer');
const cors = require('cors');

const app = express();
app.use(express.json())
//This is added to avoid cors error while using live server extension
app.use(cors());
dotenv.config({
  path: "./.env",
});

const db = mysql.createConnection({
  host: process.env.DATABASE_HOST,
  user: process.env.DATABASE_USER,
  password: process.env.DATABASE_PASS,
  database: process.env.DATABASE,
});

db.connect((err) => {
  if (err) {
    console.log(err);
  } else {
    console.log("MySQL Connection Success");
  }
});

app.use(cookieParser());
app.use(express.urlencoded({ extended: false }));

const location = path.join(__dirname, "./public");
app.use(express.static(location));
app.set("view engine", "hbs");

const partialsPath = path.join(__dirname, "./views/partials");
hbs.registerPartials(partialsPath);
```

```
// Puppeteer and Express API route
app.post('/predict', async (req, res) => {
  console.log(req.body.imageUrl);
  const itemName = await scrape(req.body.imageUrl);
  res.send({results:itemName});
  res.end();
});

async function scrape(imageUrl) {
  const browser = await puppeteer.launch();
  const page = await browser.newPage();
  await page.goto(`https://lens.google.com/uploadbyurl?url=${imageUrl}`);
  const links = await page.evaluate(() => {
    const elements = Array.from(document.querySelectorAll('span.LzliJc'));
    return elements.map(element => element.innerText.trim());
  });
  console.log(links);
  console.log("the food name is",links);
  await browser.close();
  return links
}

// Routes
app.use("/", require("./routes/pages"));
app.use("/auth", require("./routes/auth"));

app.listen(5000, () => {
  console.log("Server Started @ Port 5000");
});
```

Under Controller Folder Create Node js file named user.js

```
const mysql = require("mysql");
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const { promisify } = require("util");

const db = mysql.createConnection({
  host: process.env.DATABASE_HOST,
  user: process.env.DATABASE_USER,
  password: process.env.DATABASE_PASS,
  database: process.env.DATABASE,
});

exports.login = async (req, res) => {
  try {
    const { email, password } = req.body;
    if (!email || !password) {
      return res.status(400).render("login", {
        msg: "Please Enter Your Email and Password",
        msg_type: "error",
      });
    }

    db.query(
      "select * from login where email=?",
      [email],
      async (error, result) => {
        console.log(result);
        if (result.length <= 0) {
          return res.status(401).render("login", {
            msg: "Please Enter Your Email and Password",
            msg_type: "error",
          });
        } else {
          if (!(await bcrypt.compare(password, result[0].PASS))) {
            return res.status(401).render("login", {
              msg: "Please Enter Your Email and Password",
              msg_type: "error",
            });
          } else {
            const id = result[0].NAME;
            const token = jwt.sign({ name: id }, process.env.JWT_SECRET, {
              expiresIn: process.env.JWT_EXPIRES_IN,
            });
```

```
            console.log("The Token is " + token);
            const cookieOptions = {
              expires: new Date(
                Date.now() +
                  process.env.JWT_COOKIE_EXPIRES * 24 * 60 * 60 * 1000
              ),
              httpOnly: true,
            };
            res.cookie("joes", token, cookieOptions);
            res.status(200).redirect("/html/index.html");
          }
        }
      }
    );
  } catch (error) {
    console.log(error);
  }
};
exports.register = (req, res) => {
  console.log(req.body);
  /*
  const name = req.body.name;
  const email = req.body.email;
  const password = req.body.password;
  const confirm_password = req.body.confirm_password;
  console.log(name);
  console.log(email);
    //res.send("Form Submitted");
  */
  const { name, email, password, confirm_password } = req.body;
  db.query(
    "select email from login where email=?",
    [email],
    async (error, result) => {
      if (error) {
        confirm.log(error);
      }

      if (result.length > 0) {
        return res.render("register", {
          msg: "Email id already Taken",
          msg_type: "error",
        });
      } else if (password !== confirm_password) {
        return res.render("register", {
```

```
          msg: "Password do not match",
          msg_type: "error",
        });
      }
      let hashedPassword = await bcrypt.hash(password, 8);
      //console.log(hashedPassword);

      db.query(
        "insert into login set ?",
        { name: name, email: email, pass: hashedPassword },
        (error, result) => {
          if (error) {
            console.log(error);
          } else {
            //console.log(result);
            return res.render("register", {
              msg: "User Registration Success",
              msg_type: "good",
            });
          }
        }
      );
    }
  );
};

exports.isLoggedIn = async (req, res, next) => {
  //req.name = "Check Login....";
  //console.log(req.cookies);
  if (req.cookies.joes) {
    try {
      const decode = await promisify(jwt.verify)(
        req.cookies.joes,
        process.env.JWT_SECRET
      );
      //console.log(decode);
      db.query(
        "select * from login where NAME=?",
        [decode.name],
        (err, results) => {
          //console.log(results);
          if (!results) {
            return next();
          }
          req.user = results[0];
```

```
        return next();
      }
    );
  } catch (error) {
    console.log(error);
    return next();
  }
 } else {
  next();
 }
};

exports.logout = async (req, res) => {
 res.cookie("joes", "logout", {
  expires: new Date(Date.now() + 2 * 1000),
  httpOnly: true,
 });
 res.status(200).redirect("/");
};
```

Under js Folder Create javascript file named food.js

```
async function predictFood(url) {
    // Headers
    let results = [];
    const headers = new Headers({
        'Content-Type': 'application/json'
    });

    // Request options
    const requestOptions = {
        method: 'POST',
        headers: headers,
        body: JSON.stringify({ imageUrl: url })
    };

    //Getting the results from server by making API call
    const response = await fetch('http://localhost:5000/predict', requestOptions);
    const data = await response.json();
    const kname = document.getElementById('loadname');
    kname.innerHTML =`<p style="color:red;"><strong>Food Name is:
</strong><center><h4 style="color:white;">${data.results}</h4></center></p>`;
    results = data.results;

    // Sending a request to the Eddamam API
    const searchValue = results.join(', ');
    const edamamResponse = await
fetch(`https://api.edamam.com/search?q=${searchValue}&app_id=7aa516a5&app_key=dc83
6a223fb788b11ae390504d9e97ce&from=0&to=10`);
    const edamamData = await edamamResponse.json();
    return edamamData.hits;
}


function uploadFile() {
    // Get the selected file from the input element
    const fileInput = document.getElementById('file-upload');
    const file = fileInput.files[0];

    // Create a new FileReader to read the file as a URL
    const reader = new FileReader();

    // Define the callback function that will run when the FileReader finishes reading the file
    reader.onload = function (event) {
```

```
        // The FileReader has finished reading the file. The content of the file is now available as
a data URL (event.target.result).

        // Create an image element to display the uploaded image
        const img = document.createElement('img');
        img.src = event.target.result;

        // Append the image element to the image-preview div
        document.getElementById('image-preview').appendChild(img);
    };

    // Start reading the file as a URL
    reader.readAsDataURL(file);
}

document.querySelector('input[name="file"]').addEventListener('change', function (event) {
    const preview = document.getElementById('predicted-image');
    preview.src = URL.createObjectURL(event.target.files[0]);
});


function uploadImage() {
    const input = document.getElementById('file-upload');
    const outputDiv = document.getElementById('predicted-image-link');
    const recipieResults = document.querySelector('#results');

    if (input.files.length > 0) {
        const file = input.files[0];
        const formData = new FormData();
        formData.append('image', file);

        fetch('https://api.imgbb.com/1/upload?key=9ccc9e3f849fc3139b038160fb578d04', {
            method: 'POST',
            body: formData,
        })
            .then(response => response.json())
            .then(async data => {
                if (data.data && data.data.url) {
                    const imageUrl = data.data.url;
                    outputDiv.innerHTML = `<p>Image uploaded successfully! wait a few
seconds</p>`;

                    //Getting the Results from server
                    const results = await predictFood(imageUrl);
                    //If results are there then appending to the UI
```

```
            if(results.length) {
                let html = '';
                results.forEach((recipe) => {
                    html += `
                    <li class="recipe-item">
                        <div>
                            <img src="${recipe.recipe.image}" alt="${recipe.recipe.label}">
                            <h3 style="color:yellow;">${recipe.recipe.label}</h3>
                        </div>
                        <div class="recipe-details" >
                                <p><strong>Ingredients:</strong></p>
                            <ul>
                                ${recipe.recipe.ingredients.map(ingredient =>
`<li>${ingredient.text}</li>`).join('')}
                            </ul>
                            <p
style="color:red;"><strong>Calories:</strong><h4>${recipe.recipe.calories}</h4></p>
                        <div class="recipe-link">
                            <a href="${recipe.recipe.url}" target="_blank"><b>View
Recipe</b></a>
                        </div>
                        </div>
                    </li>
                    `;
                });
                recipieResults.innerHTML = html;
            }
        } else {
            outputDiv.innerHTML = '<p>Error uploading image. Please try again.</p>';
        }
    })
    .catch(error => {
        console.error('Error:', error);
        outputDiv.innerHTML = '<p>An Network error occurred. Please try again.</p>';
    });
} else {
    outputDiv.innerHTML = '<p>Please select an image to upload.</p>';
}
}
```

Under project folder Create .env file for Api Keys

DATABASE=pandi
DATABASE_HOST=localhost
DATABASE_USER=root
DATABASE_PASS=disaster
JWT_SECRET=1234
JWT_EXPIRES_IN=90d
JWT_COOKIE_EXPIRES=90
EDAMAM_API_KEY="dc836a223fb788b11ae390504d9e97ce&from=0&to=10"
EDAMAM_APP_ID="7aa516a5"
IMGBB_API_KEY="9ccc9e3f849fc3139b038160fb578d04"